附录

附录 A 建模计算核心代码

A.1 通用参数、功率平衡与绿电指标

```
from pathlib import Path
import numpy as np
import pandas as pd
from scipy.optimize import linprog

OUT = Path("outputs")
OUT.mkdir(exist_ok=True)

HOURS = np.arange(24)
BASE_LOAD = 6 * load_pu
WIND_CAP = 40
PV_CAP = 64
FEED_IN_PRICE = 0.3779
KWH_PER_MWH = 1000
NH3_INVEST_YUAN_PER_KGH2 = 60000
H2_KG_PER_KG_NH3 = 0.2
KG_PER_TON = 1000
PRODUCTIONS = [72, 63, 54, 45, 36]

tou_price = np.array([
    0.8024 if (10 <= h < 15 or 18 <= h < 21)
    else 0.6074 if (7 <= h < 10 or 15 <= h < 18 or 21 <= h < 23)
    else 0.3424
    for h in HOURS
])

def scenario_power(iw, ip):
    """iw/ip 为 0 基下标，返回当前装机下风电、光伏小时功率，单位 MW。"""
    return WIND_CAP * wind_s[:, iw], PV_CAP * pv_s[:, ip]

def split_buy_sell(total_load, wind, pv):
    """并网功率平衡：正缺口购电，正富余上网。"""
    gen = wind + pv
    buy = np.maximum(total_load - gen, 0)
    sell = np.maximum(gen - total_load, 0)
    return buy, sell, gen

def green_indicators(total_load, gen, buy, sell):
    """绿电直连三项指标。"""
    e_load = total_load.sum()
    e_gen = gen.sum()
    e_buy = buy.sum()
    e_sell = sell.sum()
    return {
        "用电量/MWh": e_load,
        "新能源/MWh": e_gen,
        "购电/MWh": e_buy,
        "上网/MWh": e_sell,
        "自发自用占比": (e_load - e_buy - e_sell) / e_gen,
        "绿电比例": (e_gen - e_sell) / e_load,
        "上网比例": e_sell / e_gen,
    }
```

```
def classify(ind):
    ok = [
        ind["自发自用占比"] > 0.6,
        ind["绿电比例"] > 0.3,
        ind["上网比例"] < 0.2,
    ]
    if all(ok):
        return "全满足"
    if not any(ok):
        return "全不满足"
    return "部分满足"
```

A.2 吨氨成本计算

```
def daily_cost_72(wind, pv, buy, sell, y, ecap=0, storage_throughput=0):
    """72 t/d 扩容设备日成本。功率单位 MW，小时电量单位 MWh。"""
    wind_cost = wind.sum() * KWH_PER_MWH * 0.15
    pv_cost = pv.sum() * KWH_PER_MWH * 0.12
    grid_cost = (buy * KWH_PER_MWH * tou_price).sum()
    feed_revenue = sell.sum() * KWH_PER_MWH * FEED_IN_PRICE

    alk_om = (20 * y).sum() * KWH_PER_MWH * 0.1
    pem_om = (20 * y).sum() * KWH_PER_MWH * 0.15
    nh3_om = (1.5 * y).sum() * KWH_PER_MWH * 0.002

    h2_design_kg_per_day = 72 * KG_PER_TON * H2_KG_PER_KG_NH3
    nh3_capex_daily = (
        h2_design_kg_per_day * NH3_INVEST_YUAN_PER_KGH2 / (30 * 360)
    )

    storage_capex_daily = ecap * KWH_PER_MWH * 1000 / (15 * 360)
    storage_om = 0.01 * storage_throughput * KWH_PER_MWH

    return (
        wind_cost + pv_cost + grid_cost - feed_revenue
        + alk_om + pem_om + nh3_om
        + nh3_capex_daily + storage_capex_daily + storage_om
    )

def daily_cost_36(wind, pv, buy, sell):
    """36 t/d 初始设备满负荷日成本。"""
    return (
        wind.sum() * KWH_PER_MWH * 0.15
        + pv.sum() * KWH_PER_MWH * 0.12
        + (buy * KWH_PER_MWH * tou_price).sum()
        - sell.sum() * KWH_PER_MWH * FEED_IN_PRICE
        + (10 * 24) * KWH_PER_MWH * 0.1
        + (10 * 24) * KWH_PER_MWH * 0.15
        + (0.75 * 24) * KWH_PER_MWH * 0.002
        + (36 * KG_PER_TON * H2_KG_PER_KG_NH3)
          * NH3_INVEST_YUAN_PER_KGH2 / (30 * 360)
    )
```

A.3 问题二：离散日产量与开停小时优化

```
def opt_discrete(wind, pv, production):
    """给定日产量，选择固定开机小时数，使日净成本最低。"""
    hours_on = int(production // 3)
```

```
    gen = wind + pv

    best = None
    best_key = None
    for combo in itertools.combinations(range(24), hours_on):
        y = np.zeros(24)
        y[list(combo)] = 1
        total_load = BASE_LOAD + 41.5 * y
        buy, sell, gen = split_buy_sell(total_load, wind, pv)
        cost = daily_cost_72(wind, pv, buy, sell, y)
        ind = green_indicators(total_load, gen, buy, sell)

        key = (cost, ind["购电/MWh"], -ind["自发自用占比"])
        if best is None or key < best_key:
            best_key = key
            best = {
                "y": y,
                "total_load": total_load,
                "buy": buy,
                "sell": sell,
                "gen": gen,
                "cost": cost,
                "ind": ind,
            }
    return best

def solve_all_scenarios_discrete():
    all_rows = []
    best_rows = []
    for iw in range(6):
        for ip in range(4):
            wind, pv = scenario_power(iw, ip)
            scene = f"W{iw+1}P{ip+1}"
            scene_rows = []
            for q in PRODUCTIONS:
                res = opt_discrete(wind, pv, q)
                row = {
                    "场景": scene,
                    "产量/t": q,
                    "吨氨成本/元": res["cost"] / q,
                    "购电/MWh": res["ind"]["购电/MWh"],
                    "上网/MWh": res["ind"]["上网/MWh"],
                    "自发自用占比": res["ind"]["自发自用占比"],
                    "绿电比例": res["ind"]["绿电比例"],
                    "上网比例": res["ind"]["上网比例"],
                    "判定": classify(res["ind"]),
                    "开机小时": ",".join(map(str, np.where(res["y"] > 0.5)[0])),
                }
                scene_rows.append(row)
                all_rows.append(row)
            best_rows.append(min(scene_rows, key=lambda r: r["吨氨成本/元"]))

    q2_all = pd.DataFrame(all_rows)
    q2_best = pd.DataFrame(best_rows)
    q2_all.to_csv(OUT / "q2_all_productions_discrete.csv", index=False, encoding="utf-8-
sig")
    q2_best.to_csv(OUT / "q2_best_by_scenario_discrete.csv", index=False,
```

```
encoding="utf-8-sig")
    return q2_all, q2_best
```

A.4 问题三：连续负荷率动态规划

```
def opt_continuous_dp(wind, pv, production, step=0.01):
    """连续负荷率优化。y_t 取 0.10-1.00，步长为 0.01。"""
    vals = np.arange(10, 101)
    target = int(round(production / 3 / step))

    options = []
    gen = wind + pv
    for h in range(24):
        opts = []
        for v in vals:
            y = v / 100
            total_load = BASE_LOAD[h] + 41.5 * y
            buy = max(total_load - gen[h], 0)
            sell = max(gen[h] - total_load, 0)
            cost = daily_cost_72(
                np.array([wind[h]]),
                np.array([pv[h]]),
                np.array([buy]),
                np.array([sell]),
                np.array([y]),
            )
            opts.append((v, cost, buy, sell))
        options.append(opts)

    dp = {0: (0, [])}
    for h in range(24):
        ndp = {}
        for prev_sum, (prev_cost, prev_path) in dp.items():
            for v, c, buy, sell in options[h]:
                s = prev_sum + v
                if s > target:
                    continue
                val = prev_cost + c
                if s not in ndp or val < ndp[s][0]:
                    ndp[s] = (val, prev_path + [v])
        dp = ndp

    if target not in dp:
        raise RuntimeError("未找到满足目标产量的连续调节方案")

    y = np.array(dp[target][1]) / 100
    total_load = BASE_LOAD + 41.5 * y
    buy, sell, gen = split_buy_sell(total_load, wind, pv)
    cost = daily_cost_72(wind, pv, buy, sell, y)
    ind = green_indicators(total_load, gen, buy, sell)

    return {
        "y": y,
        "total_load": total_load,
        "buy": buy,
        "sell": sell,
        "gen": gen,
        "cost": cost,
        "ind": ind,
```

```
    }

def solve_all_scenarios_continuous():
    rows = []
    best_rows = []
    for iw in range(6):
        for ip in range(4):
            wind, pv = scenario_power(iw, ip)
            scene = f"W{iw+1}P{ip+1}"
            scene_rows = []
            for q in PRODUCTIONS:
                res = opt_continuous_dp(wind, pv, q)
                row = {
                    "场景": scene,
                    "产量/t": q,
                    "吨氨成本/元": res["cost"] / q,
                    "购电/MWh": res["ind"]["购电/MWh"],
                    "上网/MWh": res["ind"]["上网/MWh"],
                    "自发自用占比": res["ind"]["自发自用占比"],
                    "绿电比例": res["ind"]["绿电比例"],
                    "上网比例": res["ind"]["上网比例"],
                    "判定": classify(res["ind"]),
                    "负荷率序列": ",".join(f"{x:.2f}" for x in res["y"]),
                }
                rows.append(row)
                scene_rows.append(row)
            best_rows.append(min(scene_rows, key=lambda r: r["吨氨成本/元"]))

    q3_all = pd.DataFrame(rows)
    q3_best = pd.DataFrame(best_rows)
    q3_all.to_csv(OUT / "q3_all_productions_continuous.csv", index=False,
encoding="utf-8-sig")
    q3_best.to_csv(OUT / "q3_best_by_scenario_continuous.csv", index=False,
encoding="utf-8-sig")
    return q3_all, q3_best
```

A.5 问题四：离网无储能模型

```
MIN_LOAD_RATE = 0.1
FULL_PROCESS_POWER = 41.5

def enforce_semicontinuous_load(raw_rate, min_rate=MIN_LOAD_RATE):
    """离网允许停机；开机后负荷率不低于 10%。"""
    raw_rate = np.asarray(raw_rate, dtype=float)
    y = np.zeros_like(raw_rate)
    mask = raw_rate >= min_rate
    y[mask] = np.minimum(raw_rate[mask], 1.0)
    return y

def offgrid_no_storage(wind, pv):
    gen = wind + pv
    residual = gen - BASE_LOAD
    y = enforce_semicontinuous_load(residual / FULL_PROCESS_POWER)
    curt = np.maximum(residual - FULL_PROCESS_POWER * y, 0).sum()
    base_deficit = np.maximum(-residual, 0).sum()
    production = 3 * y.sum()
    cost = daily_cost_72(wind, pv, np.zeros(24), np.zeros(24), y)
    return y, production, cost, curt, base_deficit
```

```
off_rows = []
for iw in range(6):
    for ip in range(4):
        wind, pv = scenario_power(iw, ip)
        y, production, cost, curt, deficit = offgrid_no_storage(wind, pv)
        off_rows.append({
            "场景": f"W{iw+1}P{ip+1}",
            "产量/t": production,
            "吨氨成本/元": cost / production if production > 0 else np.inf,
            "弃电/MWh": curt,
            "常规负荷缺供/MWh": deficit,
        })

off_df = pd.DataFrame(off_rows)
off_df.to_csv(OUT / "q4_offgrid_no_storage.csv", index=False, encoding="utf-8-sig")
```

A.6 问题四：储能参与离网调度

```
ETA_C = 0.9
ETA_D = 0.9
LOSS = 0.002

def _solve_storage_relaxation(wind, pv, ecap, modes):
    """给定开停分支状态后求解线性松弛。
    modes[h] = -1 表示未分支，0 表示停机，1 表示开机且 y>=10%。
    """
    n = 24
    gen = wind + pv
    m = 6 * n  # y, charge, discharge, soc, curtail, unserved_base
    c = np.zeros(m)
    c[:n] = -3
    c[n:2*n] = 1e-4
    c[2*n:3*n] = 1e-4
    c[4*n:5*n] = 1e-5
    c[5*n:] = 10000

    aeq, beq = [], []
    for h in range(n):
        row = np.zeros(m)
        row[h] = -FULL_PROCESS_POWER
        row[n + h] = -1
        row[2*n + h] = 1
        row[4*n + h] = -1
        row[5*n + h] = 1
        aeq.append(row)
        beq.append(BASE_LOAD[h] - gen[h])

    for h in range(n):
        row = np.zeros(m)
        row[3*n + h] = 1
        row[3*n + (h - 1) % n] = -(1 - LOSS)
        row[n + h] = -ETA_C
        row[2*n + h] = 1 / ETA_D
        aeq.append(row)
        beq.append(0)

    y_bounds = []
    for mode in modes:
```

```
        if mode == 0:
            y_bounds.append((0, 0))
        elif mode == 1:
            y_bounds.append((MIN_LOAD_RATE, 1))
        else:
            y_bounds.append((0, 1))

    bounds = (
        y_bounds
        + [(0, None)] * n
        + [(0, None)] * n
        + [(0, ecap)] * n
        + [(0, None)] * n
        + [(0, None)] * n
    )
    return linprog(c, A_eq=np.array(aeq), b_eq=np.array(beq), bounds=bounds,
method="highs")

def dispatch_storage_offgrid(wind, pv, ecap, max_nodes=5000):
    """离网储能调度：y_t=0 或 0.1<=y_t<=1，分支定界+linprog求解。"""
    n = 24
    root = tuple([-1] * n)
    root_res = _solve_storage_relaxation(wind, pv, ecap, root)
    if not root_res.success:
        raise RuntimeError(root_res.message)

    heap = []
    counter = itertools.count()
    heapq.heappush(heap, (root_res.fun, next(counter), root, root_res))
    best_res = None
    best_fun = np.inf
    nodes = 0

    while heap and nodes < max_nodes:
        lb, _, modes, res = heapq.heappop(heap)
        nodes += 1
        if lb >= best_fun - 1e-7:
            continue

        y = res.x[:n]
        viol = [
            h for h, val in enumerate(y)
            if val > 1e-6 and val < MIN_LOAD_RATE - 1e-6 and modes[h] == -1
        ]
        if not viol:
            best_res = res
            best_fun = res.fun
            continue

        h = max(viol, key=lambda k: min(y[k], MIN_LOAD_RATE - y[k]))
        for branch_value in (0, 1):
            child_modes = list(modes)
            child_modes[h] = branch_value
            child = _solve_storage_relaxation(wind, pv, ecap, tuple(child_modes))
            if child.success and child.fun < best_fun - 1e-7:
                heapq.heappush(heap, (child.fun, next(counter), tuple(child_modes),
child))

    if best_res is None:
```

```
        raise RuntimeError(f"未在 {max_nodes} 个分支节点内找到半连续可行解")

    x = best_res.x
    y = x[:n]
    y[np.abs(y) < 1e-7] = 0
    charge = x[n:2*n]
    discharge = x[2*n:3*n]
    soc = x[3*n:4*n]
    curtail = x[4*n:5*n]
    unserved = x[5*n:6*n]
    production = 3 * y.sum()
    throughput = charge.sum() + discharge.sum()
    cost = daily_cost_72(
        wind, pv, np.zeros(n), np.zeros(n), y,
        ecap=ecap, storage_throughput=throughput
    )
    return y, charge, discharge, soc, curtail, unserved, production, cost, throughput,
nodes
```

A.7 问题四：储能容量扫描和 24 场景调度明细导出

```
max_curt_scenario = off_df.sort_values("弃电/MWh", ascending=False).iloc[0]["场景"]
iw = int(max_curt_scenario[1]) - 1
ip = int(max_curt_scenario[3]) - 1
wind_max_curt, pv_max_curt = scenario_power(iw, ip)

storage_scan = []
for ecap in np.arange(0, 301, 1):
    out = dispatch_storage_offgrid(wind_max_curt, pv_max_curt, ecap)
    storage_scan.append({
        "储能/MWh": ecap,
        "产量/t": out[6],
        "吨氨成本/元": out[7] / out[6],
        "弃电/MWh": out[4].sum(),
        "分支节点数": out[9],
    })

storage_scan_df = pd.DataFrame(storage_scan)
storage_scan_df.to_csv(OUT / "q4_storage_scan.csv", index=False, encoding="utf-8-sig")
ecap_best = float(storage_scan_df.loc[storage_scan_df["吨氨成本/元"].idxmin(), "储能/MWh"])

all_dispatch_rows = []
summary_rows = []
for iw in range(6):
    for ip in range(4):
        scene = f"W{iw+1}P{ip+1}"
        wind, pv = scenario_power(iw, ip)
        y, charge, discharge, soc, curtail, unserved, production, cost, throughput, nodes
= (
            dispatch_storage_offgrid(wind, pv, ecap_best)
        )
        gen = wind + pv
        nh3_power = FULL_PROCESS_POWER * y

        for h in range(24):
            all_dispatch_rows.append({
                "场景": scene,
                "小时": h,
```

```
                "风电/MW": wind[h],
                "光伏/MW": pv[h],
                "风光合计/MW": gen[h],
                "常规负荷/MW": BASE_LOAD[h],
                "制氨负荷率": y[h],
                "制氨功率/MW": nh3_power[h],
                "储能充电/MW": charge[h],
                "储能放电/MW": discharge[h],
                "SOC/MWh": soc[h],
                "弃电/MW": curtail[h],
                "常规负荷缺供/MW": unserved[h],
            })

        summary_rows.append({
            "场景": scene,
            "产量/t": production,
            "吨氨成本/元": cost / production,
            "弃电/MWh": curtail.sum(),
            "常规负荷缺供/MWh": unserved.sum(),
            "分支节点数": nodes,
        })

pd.DataFrame(all_dispatch_rows).to_csv(
    OUT / "q4_storage_dispatch_all_scenarios.csv",
    index=False,
    encoding="utf-8-sig",
)
pd.DataFrame(summary_rows).to_csv(
    OUT / "q4_storage_dispatch_all_scenarios_summary.csv",
    index=False,
    encoding="utf-8-sig",
)
```

附录 B 正文未完整展开的数据文件索引

文件	内容说明	对应正文
q2_all_productions_discrete.csv	120×11；问题二：5 档日产量 × 24 场景离散开停完整结果	2.2
q2_best_by_scenario_discrete.csv	24×11；问题二：每个场景最低吨氨成本的离散方案	2.2
q2_boxplot_outliers.csv	38×14；问题二：箱线图离散点明细	2.2
q2_boxplot_outlier_summary.csv	18×10；问题二：离散点按产量和指标汇总	2.2
q2_typical_results_summary.csv	5×10；问题二（1）：典型风光场景五档日产量结果	2.1
q2_typical_open_hour_matrix.csv	5×25；问题二（1）：典型场景五档日产量开机小时矩阵	2.1
q3_all_productions_continuous.csv	120×12；问题三：5 档日产量 × 24 场景连续调节完整结果	3.1
q3_120_optimal_schemes.csv	120×36；问题三：120 个连续调节最优方案的逐小时功率和指标	3.1
q3_optimal_load_rate_by_scenario.csv	120×26；问题三：每种场景、每种日产量的 24 小时负荷率	3.1
q3_best_by_scenario_continuous.csv	24×12；问题三：每个场景最低吨氨成本连续方案	3.1
q2_q3_all_productions_compare.csv	120×27；问题二与问题三全部场景、全部产量对比	3.3
q2_q3_cost_difference_curves_by_production.csv	120×22；问题二、三吨氨成本差值持续曲线数据	3.3
q4_offgrid_no_storage.csv	24×5；问题四（1）：无储能离网各场景产量、成本、弃电和缺供	4.1
q4_min_capacity_estimates.csv	4×8；问题四（1）：能源自治装机估算结果	4.1
q4_min_capacity_binding_hours.csv	24×5；问题四（1）：逐小时功率自治紧约束时段	4.1
q4_energy_autonomy_compare_by_scenario.csv	24×7；问题四（1）：电量自治等比放大与分别优化逐场景对比	4.1
q4_storage_scan.csv	301×5；问题四（2）：最大弃电场景下 0-300 MWh 储能容量扫描	4.2
q4_storage_by_scenario.csv	24×7；问题四（2）：155 MWh 储能下各场景汇总结果	4.2
q4_storage_dispatch_all_scenarios.csv	576×13；问题四（2）：24 场景 × 24 小时有储能完整调度方案	4.2
q4_storage_dispatch_all_scenarios_summary.csv	24×6；问题四（2）：有储能调度逐场景汇总	4.2
q4_storage_improvement_by_scenario.csv	24×10；问题四（2）：储能前后产量、弃电和成本变化	4.2
q4_w4p1_storage_dispatch.csv	24×12；问题四（2）：最大弃电场景 W4P1 有储能逐小时调度	4.2
q4_grid_connected_same_storage_production.csv	24×11；问题四（3）：同产量并网对照结果	4.3
q4_offgrid_grid_cost_gap.csv	24×12；问题四（3）：离网储能与并网同产量吨氨成本差	4.3
q4_mode_summary.csv	5×4；问题四（3）：不同运行模式年度汇总	4.3

附录 C 正文图片与补充图片索引

文件	内容说明
q1_power_curves.png	问题一总负荷与风光出力组合图
q1_buy_sell_curves.png	问题一购电与上网曲线
q2_typical_open_hours_gantt.png	问题二（1）典型场景五档日产量开机小时甘特图
q2_typical_indicator_bars.png	问题二（1）典型场景三项绿电指标
q2_cost_buy_sell_distribution.png	问题二（2）吨氨成本、购电、上网分布
q2_green_indicator_distribution.png	问题二（2）绿电指标分布
q2_cost_distribution_curves_by_production.png	问题二（2）五档产量吨氨成本持续曲线
q3_representative_scenario_power_heatmaps.png	问题三代表场景连续调节方案热力图
q3_best_production_by_scenario.png	问题三各场景最优日产量
q3_cost_distribution_curves_by_production.png	问题三五档产量吨氨成本持续曲线
q2_q3_cost_difference_curves_by_production.png	问题二、三五档产量吨氨成本差值曲线
q4_offgrid_production_lollipop.png	问题四无储能离网日产量排序
q4_offgrid_cost_lollipop.png	问题四无储能离网吨氨成本排序
q4_offgrid_curtailment_rank.png	问题四无储能弃电量排序，识别最大弃电场景
q4_energy_autonomy_compare.png	问题四电量自治等比放大与分别优化对比
q4_storage_capacity_scan.png	问题四最大弃电场景储能容量扫描
q4_w4p1_storage_dispatch.png	问题四 W4P1 有储能逐小时调度
q4_storage_dispatch_w4p1.png	问题四 W4P1 代表场景有储能调度方案
q4_storage_dispatch_w5p1.png	问题四 W5P1 代表场景有储能调度方案
q4_storage_dispatch_w2p4.png	问题四 W2P4 代表场景有储能调度方案
q4_storage_dispatch_w3p3.png	问题四 W3P3 代表场景有储能调度方案
q4_storage_improvement_by_scenario.png	问题四储能对 24 场景的改善效果
q4_offgrid_grid_cost_gap.png	问题四同产量离网储能与并网成本差
q4_mode_summary_compare.png	问题四不同运行模式年度产量与成本对比

附录 D 可复现实验文件说明

文件	说明
A题第1-4题计算代码.ipynb	完整计算 notebook，包含数据录入、建模计算、结果导出和绘图。
第1-4题详细解答.md	正文解答文件，目前包含第 1-5 题文字、公式、结果表和主要图。
所有附件合并数据.txt	用户整理后的题目附件数据合并文本。
A题_txt格式数据/	各附件拆分后的文本数据。
outputs/	全部模型输出表格和图片。